On Correctness of Automatic Differentiation for Non-Differentiable Functions

Proseminar Differentiable Programming

Simon Wilhelm Schübel

12.07.2023

***Abstract*—Differentiation lies at the core of many machine-learning algorithms, and is well- supported by popular autodiff-systems, such as TensorFlow and PyTorch. Originally, these systems have been developed to compute derivatives of differentiable functions, but in practice, they are commonly applied to functions with non-differentiabilities. For instance, neural networks using ReLU define non-differentiable functions in general, but the gradients of losses involving those functions are computed using autodiff-systems in practice. This raises an obvious question: are autodiff-systems correct in any formal sense when they are applied to such non-differentiable functions? [3, 1]**

I. Introduction

Automatic differentiation is an increasingly important topic in many fields of interest. Naturally, the question whether autodiff-systems are correct arises. Although very theoretical, it is crucial to be answered positively. Especially when the functions given to those systems as input are non-differentiable. One major application of automatic differentiation is deep learning and training of neural networks, where non-differentiable functions appear frequently [4]. For example, among the simplest but widely used activation function for neural network; ReLU is not differentiable at 0 [2].

One of the main challenges of this paper is creating a class of functions, so called *PAP functions*, such that most non-differential functions given to autodiff-systems fit this classification. The next step is to show that autodiff-systems work reasonably well with these functions and they can help showing correctness.

Because there are more than one autodiff-system built to this day - TensorFlow, PyTorch - the paper tries to provide an answer to this question in general rather than one particular system.

II. Challenges

Most autodiff-systems are based on a chain rule for composition of functions, which are differentiable almost everywhere [3]. This makes it easy to suspect properties of these functions, that do not necessarily hold when a function is *only* differentiable almost everywhere, making it difficult to prove correctness of autodiff-systems. Lets, for propositions sake examine the following claim:

- If $f : X \to Y, g : Y \to \mathbb{R}^l$ and $g \circ f$ are differentiable almost everywhere and continuous, then the standard chain rule for $g \circ f$ should hold almost everywhere. In particular, if $f, g : (0,1) \to (0,1)$, then $(g \circ f)'(x_0) = g'(f(x_0)) * f'(x_0)$ for almost all $x_0 \in (0,1)$ [3, Claim 2].

The possible significance of this claim in this matter becomes apparent quickly, the more important is disproving it: Let $f(x) = \frac{1}{2}, g(y) = \text{ReLU}(y - \frac{1}{2}) + \frac{1}{2}$. Then $(g \circ f)(x) = \frac{1}{2}$. f, g are differentiable almost everywhere and Lipschitz continuous. g is however not differentiable at $x_0 = \frac{1}{2}$. So $g \circ f$ is not differentiable for all $x \in (0,1)$ when using the standard chain rule.

Disproving other, similarly deceiving claims is relevant as they could lead to the topic as a whole being marked off as trivial, or the non-differentiability as an exceptionally rare case in general.

This is only one of three claims that appear intuitive and would prove to be useful for making the case of autodiff-systems being correct on non-differentiable functions. The paper disproves all claims by providing concrete counterexamples.

III. Formalization

Before looking into the correctness of autodiff-systems, it's necessary to first define the term and introduce the tools needed. In particular, what *PAP functions* are and how *intensional derivatives* work.

A. Defining Correctness

If a function f is not differentiable at some x in its domain, it's not possible for an autodiff-system to provide a 'correct' evaluation of the derivative of f at x. Therefore the paper defines correctness of these systems as computing the correct derivative of f for almost every $x \in X$, when the function f is differentiable almost everywhere. Meaning as long as the non-differentiability of f occurs rarely, the autodiff-system should be able to handle it.

B. PAP - piecewise analytic under analytic partition

The idea is to find a so called *PAP Representation* γ of a function f. This means, for a given function f find a bunch of tuples $\gamma = \{\langle A^i, f^i \rangle\}_{i \in I}$ for some indexset I such that $f(x) = f^i(x)$ if $x \in A^i$. A^i is the respective domain of each function f^i. Notice, that because all f^i are analytic they are also infinitely differentiable. The domains A^i are called analytic as their boundaries are defined by zero sets of analytic functions.

For instance *one* possible representation of ReLU would be $\gamma_{\text{ReLU}} = \{\langle \mathbb{R}_{<0}, x \mapsto 0 \rangle, \langle \mathbb{R}_{\geq 0}, x \mapsto x \rangle\}$. Therefore showing, ReLU is PAP. This however is not the only possible representation, another one could be

$$\gamma_2 = \{\langle \mathbb{R}_{<0}, x \mapsto 0 \rangle, \langle \{0\}, x \mapsto xc \rangle, \langle \mathbb{R}_{>0}, x \mapsto x \rangle\}.$$

1) Properties: PAP functions fulfill properties, that make it easier to use them later on.

- *All PAP functions are differentiable almost everywhere.*
- *Let $f : X \to Y$ and $g : Y \to \mathbb{R}^l$ be PAP functions. $g \circ f$ is a PAP function.* - This makes it easy to show that

most - if not all - functions describing neural networks are indeed PAP functions.

C. Intensional derivative

An intensional derivative of a *PAP Representation* γ is the set $D\gamma = \{\langle A^i, Df^i\rangle\}$, where Df^i is the standard derivative of f^i viewed as a function from the domain X^i of f^i to $\mathbb{R}^{m\times n}$ [3]. An intensional derivative is equal to the regular derivative of a function almost everywhere. Also every intensional derivative is again *PAP*.
For example,

$$D\gamma_2 = \{\langle \mathbb{R}_{<0}, x \mapsto 0\rangle, \langle \{0\}, x \mapsto c\rangle, \langle \mathbb{R}_{\geq}, x \mapsto 1\rangle\}.$$

A first order intensional derivative $\partial.f$ of a function f is the set of all intensional derivatives, for all *PAP Representation*s of f: $\partial.f = \{D\gamma \mid \gamma \text{ is a PAP representation of } f\}$. Note, that all $k-$th order derivatives are not empty and agree with the k-th standard derivative almost everywhere.

IV. Correctness of autodiff-systems

A. Programming language

To generate an answer to the correctness of autodiff-systems, the paper defines a programming language. The language accepts $x_1, \ldots, x_N \in \mathbb{R}$ as input variables. Furthermore it has following syntax :
$e ::= c \mid x_i \mid \bar{f}(e_1, \ldots, e_n) \mid if(e_1 > 0)\ e_2\ e_3$. Meaning e is either a real number c, an input variable x_i, the application of a *PAP function* $\bar{f} : \mathbb{R}^n \to \mathbb{R}$ to arguments $e_1, \ldots e_n$ or a conditional expression. Each program e can therefore be interpreted as a function $[\![e]\!] : \mathbb{R}^N \to \mathbb{R}$. Because the composition of *PAP functions* is always *PAP* again $[\![e]\!]$ is always *PAP*.

B. Differentiation of a program

Because $[\![e]\!]$ can only be a composition of one of four expressions, it's possible to define the symbolic derivative of $[\![e]\!]$; $[\![e]\!]^\nabla : \mathbb{R}^N \to \mathbb{R}^{1\times N}$. The most interesting case being the conditional statement:

$$[\![if(e_1 > 0)\ e_2\ e_3]\!]^\nabla v = \\ if([\![e_1]\!]^\nabla v > 0) \text{ then } [\![e_2]\!]^\nabla v \text{ else } [\![e_3]\!]^\nabla v$$

Other cases of e are differentiated using the chain rule for differentiable functions.

C. Requirements of correctness

In order to be classified as correct by the definition given, an autodiff-system has to fulfill two requirements:

1) $\forall \bar{f} : \mathbb{R}^n \to \mathbb{R}$, the system should come up with a function $Df : \mathbb{R}^n \to \mathbb{R}^{1\times n}$, such that $Df = D\gamma$ is an intensional derivative for some *PAP Representation* γ of f.
2) Given a program e and an input $v \in \mathbb{R}^N$, the systems output should match the symbolic differentiation $[\![e]\!]^\nabla$.

If one can show that these requirements are met by an autodiff-system, then they ensure the correctness of the system.

D. Proof

Firstly, it's essential to introduce Theorem 12, which says that if $\tilde{D}f \in \partial.f$ for all primitive functions $\bar{f}$, then $[\![e]\!]^\nabla \in \partial.[\![e]\!]$ for all programs e [3, Theorem 12]. Meaning that if for all primitive functions $\bar{f}$ the derivative Df of f is an intensional derivative, the symbolic derivative $[\![e]\!]^\nabla$ is an intensional derivative of $[\![e]\!]$: $[\![e]\!]^\nabla \in \partial.[\![e]\!]$.

Now suppose an autodiff-system for the language e fulfills both requirements. Then - because of requirement 2 - we can set the output df of the system to be the symbolic derivative: $df = [\![e]\!]^\nabla$. Because of requirement 1, the prerequisites for Theorem 12 are met, which means df is an intensional derivative. If the system now performs forward-mode automatic differentiation with a tangent vector $w \in \mathbb{R}^N$ we can claim that if the autodiff-system performs forward-mode:

$$df(v) * w \in \mathbb{R}$$

for a tangent vector $w \in \mathbb{R}^N$ evaluates to the jacobian-vector product for every input vector $v \in \mathbb{R}^N$. Recall that the k-th intensional derivative matches the k-th standard derivative almost everywhere. Because of this, $df(v) * w$ matches with the jacobian-vector product of the standard derivative of $[\![e]\!]$ for almost all inputs $v \in \mathbb{R}^N$ henceforth proofing correctness for the system as defined.

References

[1] Martín Abadi et al. *TensorFlow: A system for large-scale machine learning*. 2016. arXiv: 1605.08695 [cs.DC].

[2] Abien Fred Agarap. *Deep Learning using Rectified Linear Units (ReLU)*. 2019. arXiv: 1803.08375 [cs.NE].

[3] Wonyeol Lee et al. *On Correctness of Automatic Differentiation for Non-Differentiable Functions*. 2020. arXiv: 2006.06903 [cs.LG].

[4] Gian Paolo Leonardi and Matteo Spallanzani. *Analytical aspects of non-differentiable neural networks*. 2020. arXiv: 2011.01858 [cs.LG].